**Academic Year: 2026-27**
**Semester: VII**
**Class / Branch/ Div: BE- IT B**
**Subject: SAD Lab**
**Name of Instructor: Prof Ganesh Gourshete**

**Name of Student: Sonika Sawant**
**Student ID: 23104054**
**Roll No.: 65**
**Date of Submission: 21/7/26**

# Experiment No:2

**Aim: To study SDLC and implement Threat Modeling.**

**Theory:**

1. **Secure Software Development Lifecycle (SSDLC)**

- How does a secure SDLC work?

A Secure SDLC requires adding security testing at each software development stage, from design, to development, to deployment and beyond. Examples include designing applications to ensure that your architecture will be secure, as well as including security risk factors as part of the initial planning phase.

- Why Is Secure SDLC Important?

Secure SDLC is important because application security is important. The days of releasing a product into the wild and addressing bugs in subsequent patches are gone. Developers now need to be cognizant of potential security concerns at each step of the process. This requires integrating security into your SDLC in ways that were not needed before. As anyone can potentially gain access to your source code, you need to ensure that you are coding with potential vulnerabilities in mind. As such, having a robust and secure SDLC process is critical to ensuring your application is not subject to attacks by hackers and other nefarious users.

- History of SDLC Practices

Software Development Lifecycle (SDLC) describes how software applications are built. It usually contains the following phases:

- Requirements gathering
- Design of new features based on the requirements
- Development of new capabilities (writing code to meet requirements)
- Verification of new capabilities—confirming that they do indeed meet the

requirements

- Maintenance and evolution of these capabilities once the release goes out the door

The Waterfall model is one of the earliest and best-known SDLC methodologies, which laid the groundwork for these SDLC phases. Developed in 1970, these phases largely remain the same today, but there have been tremendous changes in software engineering practices that have redefined how software is created.

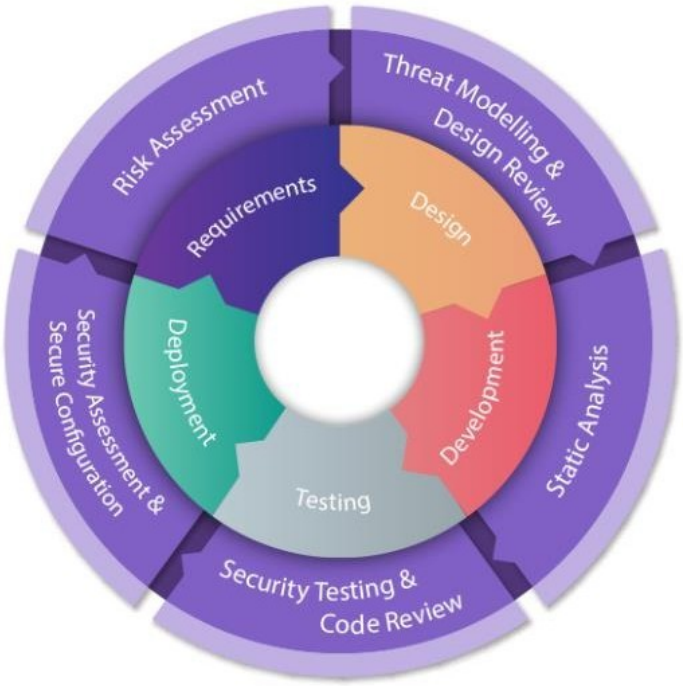


Fig. Secure Software Development Lifecycle (SSDLC)

2. **What is Threat Modelling?**
   Threat modelling is a method of optimizing network security by locating vulnerabilities, identifying objectives, and developing countermeasures to either prevent or mitigate the effects of cyber-attacks against the system.

While security teams can conduct threat modelling at any point during development, doing it at the start of the project is best practice. This way, threats can be identified sooner and dealt with before they become an issue.

It's also important to ask the following questions:

- What kind of threat model needs building? The answer requires studying data flow transitions, architecture diagrams, and data classifications, so you get a virtual model of the network you're trying to protect.
- What are the pitfalls? Here is where you research the main threats to your network and applications.
- What actions should be taken to recover from a potential cyberattack? You've identified the problems now; it's time to figure out some actionable solutions.
- Did it work? This step is a follow-up where you conduct a retrospective to monitor the quality, feasibility, planning, and progress.

Threat modelling is a core element of the **Microsoft Security Development Lifecycle (SDL)**. It’s an engineering technique you can use to help you identify threats, attacks, vulnerabilities, and countermeasures that could affect your application. You can use threat modelling to shape your application's design, meet your company's security objectives, and reduce risk.

**There are five major threat modelling steps:**

- Defining security requirements.

- Creating an application diagram.
- Identifying threats.
- Mitigating threats.
- Validating those threats have been mitigated.

Threat modelling should be part of your routine development lifecycle, enabling you to progressively refine your threat model and further reduce risk.

The Threat Modelling Tool enables any developer or software architect to:

- Communicate about the security design of their systems.
- Analyse those designs for potential security issues using a proven methodology.
- Suggest and manage mitigations for security issues.

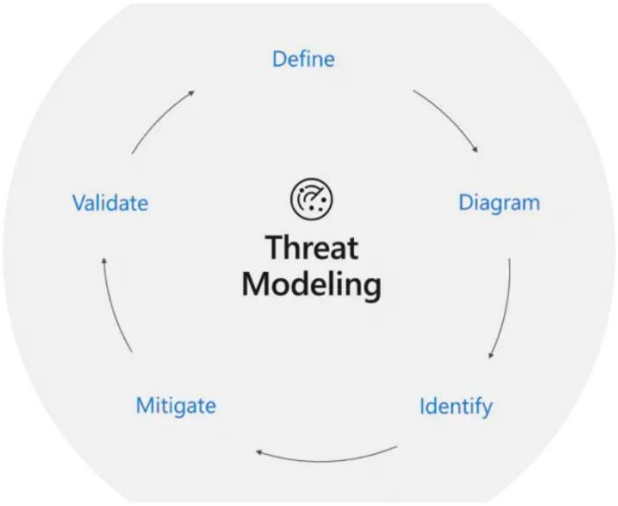


Fig. Steps in Threat Modelling

**Threat Modelling Process**

Threat modelling consists of defining an enterprise's assets, identifying what function each application serves in the grand scheme, and assembling a security profile for each application. The process continues with identifying and prioritizing potential threats, then documenting both the harmful events and what actions to take to resolve them.

Or, to put this in lay terms, threat modelling is the act of taking a step back, assessing your organization's digital and network assets, identifying weak spots, determining what threats exist, and coming up with plans to protect or recover.

It may sound like a no-brainer, but you'd be surprised how little attention security gets in some sectors. We're talking about a world where some folks use the term PASSWORD as their password or leave their mobile devices unattended. In that light, it's hardly surprising that many organizations and businesses haven't even considered the idea of threat modelling.

**Threat Modelling Methodologies**

There are as many ways to fight cybercrime as there are types of cyber-attacks. For instance, here are ten popular threat modelling methodologies used today.

**1. STRIDE**

A methodology developed by Microsoft for threat modelling, it offers a mnemonic for identifying security threats in six categories:

- Spoofing: An intruder posing as another user, component, or other system feature that contains an identity in the modelled system.
- Tampering: The altering of data within a system to achieve a malicious goal.

- Repudiation: The ability of an intruder to deny that they performed some malicious activity, due to the absence of enough proof.
- Information Disclosure: Exposing protected data to a user that isn't authorized to see it.
- Denial of Service: An adversary uses illegitimate means to exhaust services needed to provide service to users.
- Elevation of Privilege: Allowing an intruder to execute commands and functions that they aren't allowed to.

**2. DREAD**

Proposed for threat modelling, but Microsoft dropped it in 2008 due to inconsistent ratings. OpenStack and many other organizations currently use DREAD. It's essentially a way to rank and assess security risks in five categories:

- Damage Potential: Ranks the extent of damage resulting from an exploited weakness.
- Reproducibility: Ranks the ease of reproducing an attack
- Exploitability: Assigns a numerical rating to the effort needed to launch the attack.
- Affected Users: A value representing how many users get impacted if an exploit becomes widely available.
- Discoverability: Measures how easy it is to discover the threat.

**3. P.A.S.T.A**

This stands for Process for Attack Simulation and Threat Analysis, a seven-step, risk-centric methodology. It offers a dynamic threat identification, enumeration, and scoring process. Once experts create a detailed analysis of identified threats, developers can develop an asset-centric mitigation strategy by analysing the application through an attacker-centric view.

**4. Trike**

Trike focuses on using threat models as a risk management tool. Threat models, based on requirement models, establish the stakeholder-defined "acceptable" level of risk assigned to each asset class. Requirements model analysis yields a threat model where threats are identified and given risk values. The completed threat model is then used to build a risk model, factoring in actions, assets, roles, and calculated risk exposure.

**5. VAST**

Standing for Visual, Agile, and Simple Threat modelling, it provides actionable outputs for the specific needs of various stakeholders such as application architects and developers, cybersecurity personnel, etc. VAST offers a unique application and infrastructure visualization plan so that the creation and use of threat models don't require any specialized expertise in security subject matters.

**6. Attack Tree**

The tree is a conceptual diagram showing how an asset, or target, could be attacked, consisting of a root node, with leaves and children nodes added in. Child nodes are conditions that must be met to make the direct parent node true. Each node is satisfied only by its direct

child nodes. It also has "AND" and "OR" options, which represent alternative steps taken to achieve these goals.

**7. Common Vulnerability Scoring System (CVSS)**

This method provides a way to capture a vulnerability's principal characteristics and assigning a numerical score (ranging from 0-10, with 10 being the worst) showing its severity. The score is then translated into a qualitative representation (e.g., Low, Medium, High, and Critical). This representation helps organizations effectively assess and prioritize their unique vulnerability management processes.

**8. T-MAP**

T-MAP is an approach commonly used in Commercial Off the Shelf (COTS) systems to calculate attack path weights. The model incorporates UML class diagrams, including access class, vulnerability, target assets, and affected value.

**9. OCTAVE**

The Operationally Critical Threat, Asset, and Vulnerability Evaluation (OCTAVE) process is a risk-based strategic assessment and planning method. OCTAVE focuses on assessing organizational risks only and does not address technological risks. OCTAVE has three phases:

- Building asset-based threat profiles. (Organizational evaluation)
- Identifying infrastructure vulnerabilities. (Information infrastructure evaluation)
- Developing and planning a security strategy. (Evaluation of risks to the company's critical assets and decision making.)

**10. Quantitative Threat Modelling Method**

This hybrid method combines attack trees, STRIDE, and CVSS methods. It addresses several pressing issues with threat modelling for cyber-physical systems that contain complex interdependencies in their components. The first step is building components attack trees for the STRIDE categories. These trees illustrate the dependencies in the attack categories and low-level component attributes. Then the CVSS method is applied, calculating the scores for all the tree's components.

There are several ways to assess security threats, which is great as the threats are real and will continue as hackers develop new ways to conduct their dark activities.

**Conclusion:** Threat modelling is a structured activity for identifying and evaluating application threats and vulnerabilities. So, we should start the threat modelling activity early in the architecture and design phase of your application life cycle, and then continually update and refine the model as you learn more about your design and implementation. Threat modelling can be used to shape your application design to meet your security objectives, to help weigh the security threat against other design concerns when making key engineering decisions, and to reduce the risk of security issues arising during development and operations.